```
#include <atomic>
#include <thread>

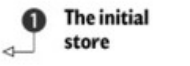
std::vector<int> queue_data;
std::atomic<int> count;

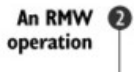
void populate_queue()
{
    unsigned const number_of_items=20;
    queue_data.clear();
    for(unsigned i=0;i<number_of_items;++i)
    {
        queue_data.push_back(i);
    }

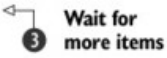
    count.store(number_of_items,std::memory_order_release);
}

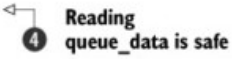

void consume_queue_items()
{
    while(true)
    {
        int item_index;
        if((item_index=count.fetch_sub(1,std::memory_order_acquire))<=0)
        {
            wait_for_more_items();
            continue;
        }
        process(queue_data[item_index-1]);
    }
}

int main()
{
    std::thread a(populate_queue);
    std::thread b(consume_queue_items);
    std::thread c(consume_queue_items);
    a.join();
    b.join();
    c.join();
}
```









To see what this means and why it's important, consider an `atomic<int>` being used as a `count` of the number of items in a shared queue, as in the following listing.

One way to handle things would be to have the thread that's producing the data store the items in a shared buffer and then do `count.store(number_of_items,memory_order_release)` ❶ to let the other threads know that data is available. The threads consuming the queue items might then do `count.fetch_sub(1,memory_order_acquire)` ❷ to claim an item from the queue, prior to actually reading the shared buffer ❹. Once the `count` becomes zero, there are no more items, and the thread must wait ❸.

If there's one consumer thread, this is fine; the `fetch_sub()` is a read, with `memory_order_acquire` semantics, and the store had `memory_order_release` semantics, so the store synchronizes-with the load and the thread can read the item from the buffer. If there are two threads reading, the second `fetch_sub()` will see the value written by the first and not the value written by the `store`. Without the rule about the release sequence, this second thread wouldn't have a happens-before relationship with the first thread, and it